

Embedded Computing Systems

Component Based Software Design

Re-implement the BuildCore scripts using Makefile

2021 / 2022

Author:
Veronica Rispo

Contents

1 Introduction to Component Based Software Design **2**

2 Hierarchical scheduling for components **3**
2.1 Real-time guarantees in CBSD 3

3 Project: Re-implement the BuildCore scripts using a Makefile **5**
3.1 Introduction 5
3.2 BuildCore 5
3.3 Project 5
3.3.1 Makefile 6
3.4 Test 9

1 Introduction to Component Based Software Design

Component Based Software Design is process of designing or developing complex software systems by connecting many smaller components.
The characteristic properties of a component are that in:

- is a unit of independent deployment : it needs to be well separated from its environment and other components.
- is a unit of third-party composition : to be composable by third party it needs to be self-contained and come with clear specification of what it requires and provides by means of well-defined interfaces.
- has no (externally) observable state : it is required that the component cannot be distinguished from copies of its own.

An example of complex system composed by many components that are connected each others using their well specified interface is shown in Figure 1.

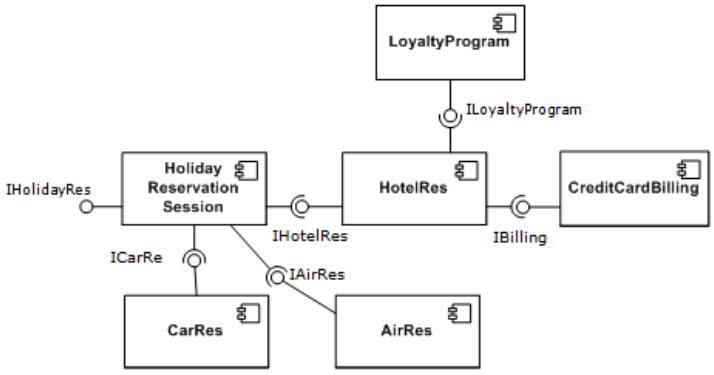


Figure 1: Component based software system example

2 Hierarchical scheduling for components

The components run on different physical nodes or are isolated using a virtual machine and can use even different operating systems. Component interfaces focus on the functional aspects regarding non-functional aspects like the temporal behaviour we can assume that component respect some temporal constraints, provides by real-time guarantees.
In general a component C^i is composed by n^i real-time tasks. The component is modelled as a set of real-time tasks : $C^i = \{(C_0^i, D_0^i, T_0^i), .., (C_n^i, D_n^i, T_n^i)\}$
To schedule the components and combine their temporal guarantees a smart solution is use the hierarchical scheduling.
We have 2-levels hierarchy (shown in Figure 2):

- **Global scheduler** : assign CPU to components without seeing the components tasks
- **Local scheduler** : assign CPU to single tasks. Every component is free to define their own scheduler

A possible implementation is a virtual machine per component where the global scheduler is the host/hypervisor scheduler and the local scheduler is the guest scheduler.

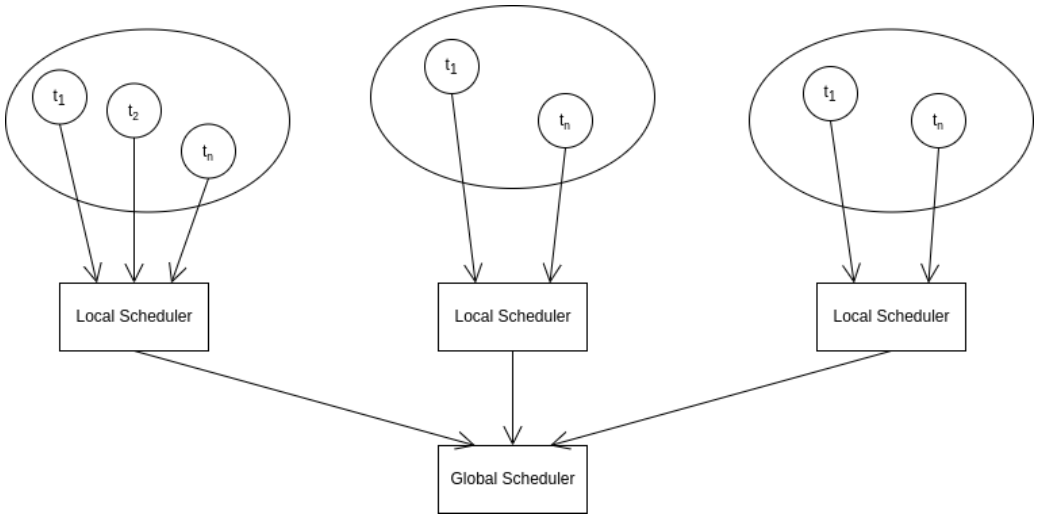


Figure 2: Hierarchical scheduling - 2-level hierarchy

2.1 Real-time guarantees in CBSD

The global or root scheduler must:

- provide some kind of temporal protection between components to analyse the schedulability of a component independently from the others.

– guarantee that each component receive a minimum amount of resource (CPU time) in a time interval

An algorithm that guarantees this properties is the **Hard Constant Bandwidth Server**, a modified version of the Constant Bandwidth Server, that is implemented in the **SCHED_DEADLINE** of linux.
Suppose to know the amount of time needed by a component to respect its temporal constraint (demanded time) and the amount of time provided by the global scheduler (supplied time). A component is **schedulable** if *demanded time* $\leq$ *supplied time* .

3 Project: Re-implement the BuildCore scripts using a Makefile

3.1 Introduction

As asserted before the various components composing a system can be executed inside a virtual machine (VM) , for example using QEMU/KVM. The VM is started using a bash script like this:

```
qemu-system-x86_64 -initrd min_core.gz  -machine pc,accel=kvm -smp cpus=1
    -kernel bzImage -append "sleeptime=1" &
PID=$!
echo PID: $PID
echo vCPU: $((PID+2))
```

where:

- "bzImage" : is a compiled Linux kernel image
- machine pc : means that I want emulate a standard pc
- kvm : I want to use this kernel module for hw assisted vistualization
- min_core.gz : is an initramfs containing the minimum amount of files needed to boot the system and eventually a test application (in a real system this application will be replaced by a real component)

An initramfs is a pseudo-filesystem decompressed in RAM at boot time. The compression format is gzip's one (gz) and the archive is stores in the "newc" variant of the "cpio" format. The advantage is that the gzipped cpio archive can be concatenated, so an initramfs can be genereted by independently generating a minimal initramfs "core" and the filesystem containing the component.

3.2 BuildCore

The BuildCore (bash scripts) package generate the "core" initramfs based on busybox and some custom scripts (/etc/init.d/rcS). To use the BuildCore you have to create a empty directory and run "buildcore.sh" inside it passing as a parameter the absolute path of the desired output file.

3.3 Project

The BuildCore package contains a set of scripts and other files needed to build the initramfs the aim of this project is re-implement those scripts using a Makefile.
Instead of using the imperative approach of the shell script it is used a declarative approach used by Makefiles. The advantage is automatic tracking of the dependencies.
Similar to BuildCore to use the Project you have to create an empty directory and run the make command inside it in this way:

```
make -f PATH_SOURCE_DIR/Makefile FILE_NAME="min_core_name" S_DIR="PATH_SOURCE_DIR"
```

with:

- **PATH_SOURCE_DIR** : path relative or absolute where the project files and directory are (default value: ../Progetto)
- **min_core_name** : name wanted per the initramfs generated (default value: minimal_core)

For example:

```
make -f ../SourcesDir/Makefile S_DIR="../SourcesDir" FILE_NAME="min_core_name"
```

3.3.1 Makefile

The final target of the Makefile is the core initramfs (FILE_NAME.gz). To build that I need to have sudo and the necessary library related to it and busybox with its necessary library.

After setting the variable useful for downloading and compiling busybox and sudo,opzionally if the ARCH and PREFIX variable are setted when the makefile is called are setted more specific variable for the compilation and in the end we exported the variables that are needed in the other makefiles.

```
BBVER := 1.29.3
GLIBCVER := 2.28
KVER := 4.28.16
SUDOVER := 1.8.26
SUDO_PATH := dist #sudover x.y.z se y = 8 sudo_path = dist else =dist/old
FILE_NAME ?= minimal_core
DIR_NAME := $(shell pwd)
S_DIR ?= ../Progetto
CC := gcc

CPUS := $(shell grep processor /proc/cpuinfo | wc -l)
ifneq ($(CPUS),1)
CPUS := $(shell expr $(CPUS) - 1)
endif

ifeq (x$(ARCH),xx86_64)
EXTRANAME := 64
CFLAGS := -m64
LDFLAGS := -m64
        ifneq ($(arch),x86_64)
CROSS :="--host=x86_64-unknown-linux-gnu"
        endif
endif

ifeq (x$(ARCH),xx86)
EXTRANAME := 32
CFLAGS := -m32
LDFLAGS := -m32
```

```
        ifneq ($(arch),x86_64)
CROSS := "--host=i686-unknown-linux-gnu"
        endif
endif

ifeq (x$(ARCH),xmusl)
EXTRANAME := musl
        ifeq ($(arch),x86_64)
CROSS :="--host=x86_64-unknown-linux-gnu"
MARCH :="x86_64-linux-musl"
        else
CROSS := "--host=i686-unknown-linux-gnu"
MARCH := "i686-linux-musl"
        endif
MCROSS :=$(MUSL)/bin/$(MARCH)-
endif

#Adjust the cross-compilation variables if exists
ifneq (x$(PREFIX),x)
MCROSS :=$(PREFIX)-
CROSS :="--host=$(PREFIX)"
ARCH :=$(shell $(PREFIX)-gcc- -dumpmachine | cut -d '-' -f 1)
endif

export CC , EXTRANAME , DIR_NAME , MCROSS , SUDOVER , BBVER , CPUS , S_DIR
```

For the target that have to execute commands in a different directory respect to the main one, for sake of simplicity and to make the code more readable it is called another makefile that is execute in the right directory. In fact when a cd command, or more in general a command, is executed in a makefile this is performed in a sub-process shell, and affects neither make nor the shell you're working from. To be executed is the same sub-process shell all the command that follow the cd must be in the same line divided by a semi-colon. Note that you cannot use new lines as they are interpreted by make as the end of the rule, so any new lines you use for clarity need to be escaped by a backslash.

```
bb_build-$(BBVER)$(EXTRANAME)/_install/bin/sudo:
    bb_build-$(BBVER)$(EXTRANAME)/_install/lib/libgcc_s.so.1 sudo_build-$(SUDOVER)$(EXTRANAME)
        cd sudo_build-$(SUDOVER)$(EXTRANAME) && $(MAKE) -f ../$(S_DIR)/installsudo.mk
        @echo "        done!"
```

The first thing to do is download and decompress busybox and then we need to build and install in the sub-directory bb_build-$(BBVER)$(EXTRANAME) so we call the *buildandinstalbb.mk* makefile that simply do that.

Before moving on to the part about sudo you have to build root, even in this case I have to work in the sub directory bb_build-$(BBVER)$(EXTRANAME) calling the *buildingroot.mk* makefile. This

one is more complex in fact it use a recursive function to find the shared library needed by busybox using readelf on the file **_install/bin/busybox** and after finding the directory where those libraries are copying it in the lib directory created in _install. Then fetch the standard library too.

```
define search_file
$(eval found:="NO")
$(foreach E,$(1), $(if $(filter x$(E),x$(2)),$(eval found:="YES")))
endef

define do_ldd
$(eval L:=$(2))
$(eval FILES := $(shell readelf -d $(1) | grep $(L_cmd) | cut -d '[' -f 2 | cut -d ']' -f 1))
$(foreach F, $(FILES),\
$(call search_file,$(L),$(shell "$(MCROSS)"$(CC) -print-file-name=$(F))) \
$(if $(filter $(found),"NO"), \
$(eval L:=$(L) $(shell "$(MCROSS)"$(CC) -print-file-name=$(F))) \
$(call do_ldd,$(shell "$(MCROSS)"$(CC) -print-file-name=$(F)),$(L)) ) \
)
endef
```

Note that to select the shared library in the libraries given by the readelf I have to use the grep command on the result, but it depend from the language of the result, in this case are supported only two language english and italian (check the laguage and use the word "shared" or "condivisa")

```
L_cmd := $(shell echo $$LANG)
L_cmd := $(shell echo $(L_cmd) | cut -d '_' -f 1)
ifeq ($(L_cmd),it)
L_cmd := condivisa
else
L_cmd := Shared
endif
```

Now we need to work on the sudo part, like for busybox first we have to download and uncompress sudo. After that we build sudo in the sub-directory sudo_build-$(SUDOVER)$(EXTRANAME) using the *buildsudo.mk* makefile.

Finally I have to install sudo moving in the same sub-directory sudo_build-$(SUDOVER)$(EXTRANAME)and using the *installsudo.mk* makefile. This act in the same way seen in the buildroot makefile but analyse this file **_install/bin/sudo** to find the shared libraries. Then fetch the libnss libraries.

Now I have all the necessary thing to generate the initramfs.

```
$(FILE_NAME).gz: bb_build-$(BBVER)$(EXTRANAME)/_install/bin/sudo
    cd bb_build-$(BBVER)$(EXTRANAME)/_install && $(MAKE) -f ../../$(S_DIR)/finalfixups.mk
    @echo "        done final fixups!"
    @echo -n "MkInitRAMFs bb_build-$(BBVER)$(EXTRANAME)/_install $(DIR_NAME)/$(FILE_NAME)"
```

```
cd bb_build-$(BBVER)$(EXTRANAME)/_install ; \
find . | cpio -o -H newc | gzip > $(DIR_NAME)/$(FILE_NAME).gz
```

The *finalfixup.mk* makefile simply copies the libraries present in the lib64 folder into lib and then deletes and recreates it as a shortcut to lib.

3.4 Test

Now in the directory where is launched the makefile I have the requested initramfs, now I can use it to launch a virtual machine (VM) using QEMU/KVM using the script seen before in 3.1 Introduction. The result is shown in Figure 3.

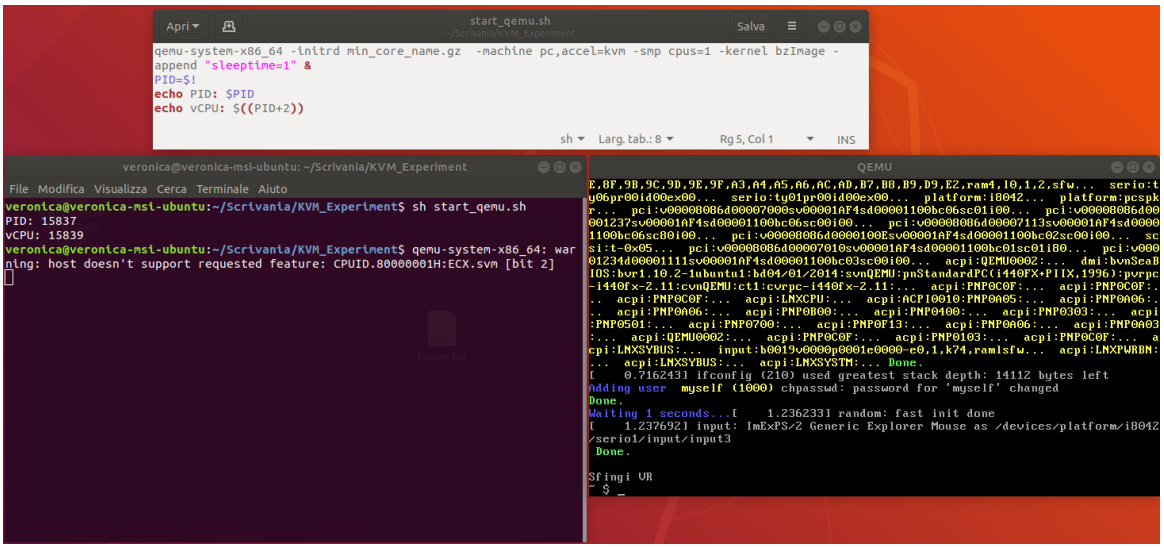


Figure 3: VM that use the generated initramfs